Go Game Project

Hoàng Nguyên Anh, Nguyễn Phan Minh Nhật

Last edited day: November 24, 2025

Overview

The project aims to develop a Go game application that is smooth and lightweight in size and performance, while still providing a decent gaming experience and point-counting aid to the players.

Contents

Chapter 1

Requirement

1.1 Game modes

- 2-player mode
  - Two players can play against each other on the same device.
  - The game manages turns automatically, alternating between Player 1 (white) and Player 2 (black).

1.2 User Interface (UI)

- A 19x19 Go board with placeable stones
- A start menu that renders before a game is started
  - A “New Game” button that starts the game in two-player mode.
  - A “Load Game” button to load saved unfinished games from storage.
  - A “Settings” button to personalize the UI, as well as the audio to the liking of the player.
  - An “Exit” button to exit the game.
- An in-game sidemenu is rendered to the right of the board.
  - A Turn Indicator on the top.
  - A “Pass Move” button to pass current move.
  - A “Start New Game” to begin a new game in the same mode.
  - A “Reset Game” to reset the current game in the same mode.
  - “Undo Move” and “Redo Move” buttons to undo and redo the latest string of moves.
  - “Save Game” and “Load Game” buttons to access the Save and Load menus.
  - A “Settings” button to access the Settings menu in-game.
  - A “Exit” button to return to the main menu.
- A Load menu displays buttons to load saved games, each of which shows previews of the board when being clicked on and loads the actual game when the button “Continue Playing” is clicked. There is also a clear button for each saved file to delete that saved game.

- A Save menu displays buttons to save current game. One clicked on, each button display a preview of the saved game that is going to be overwritten if the file is already occupied. There is also a clear button for each saved file to delete that saved game.

- A “Settings” menu with buttons to allow toggling board theme as well as stone styles, mute and unmute sound effects and background music, change the currently playing background music, and adjust audio volume. Press “Back” to go back to the currently playing game or main menu.

1.3 Go board representation

- Go board is drawn using SFML graphic libraries and background images.

- Pieces are placed by clicking on the desired intersection.

- Legal moves are rendered as ghost stones when the player hovers the mouse over the intersection.

- Illegal moves (including movement to intersections that are not legal, by suicide rule, superko rule, etc) results in an “invalid move” sound effect and do not count as a move.

- Moves are managed automatically, alternating between two colors after each move. Current player’s turn is displayed by an indicator on the top portion of the screen.

- Several metrics are displayed at the bottom right of the sidebar: move notification, move validity status, number of consecutive passes, and capture status.

1.4 Game Logic

- Pieces’ movement: After each state change, the board’s current state is processed to create a grid of legal and illegal next moves.

- Possible illegal actions are reviewed in the process, including overlap of stones, superko violation, or suicidal moves.

- In case of two consecutive passes, a pop-up menu is displayed to inform players that Life and Death assessment phase has begun. Players can click on stones to toggle between life and death.

- If an agreement is reached between two players, the score is calculated; otherwise the game continues.

1.5 Game State Management & Logic

1.5.1 System Overview

The game state is managed through a hybrid data structure approach, utilizing a 2D Grid Representation for immediate board access and a Move History for state tracking.

- **Board Representation:** A 2D array maintains the current status of each intersection (Empty, Black, or White), enabling $O(1)$ access for rendering and logic checks.

- **Move History:** A sequence of past moves is recorded to enforce the *Ko Rule* (preventing infinite loops by repeating board states) and to facilitate the Undo/Redo mechanism.

1.5.2 Turn Management

Turn logic is encapsulated within the `GoGame` controller. The system employs an event-driven flow: strictly after a move is validated as legal by the `GoEngine` and committed to the board data structure, the control flow automatically yields to the opposing player.

1.5.3 Undo/Redo Architecture

The system implements the command pattern using one structure of vectors. Boundary checks are performed before any pop operation to prevent stack overflow or underflow errors.

- **Move data vectors:** Stores the history of previous game states (board states, Zobrist hashes of each board state and number of consecutive passes). Upon executing a new move, the current state is pushed onto this stack.

Logic Flow: The top of the vectors always represents the active game state. Whenever a **new** move is placed on the board, the data with move number higher or equal to the current move is flushed to maintain linear history consistency.

1.5.4 Game Initialization

Upon startup, the system initializes the grid with `Empty` values and resets both history stacks, preparing the `GoEngine` for the first Black move.

1.5.5 Key Data Structures

- `std::vector<std::vector<Player>> board`: Represents the 19×19 grid coordinate system.
- This struct represents all states from the beginning of the 19×19 grid coordinate system:

```
std::struct History{
    std::vector<std::vector<std::vector<Player>>> board;
    std::vector<uint64_t> cur_hash;
    std::vector<int> consecutivePass;
}
```

- `std::unordered_map<uint64_t,int>superkoMap` : Stores all hashes representing the game state that already exists for easy access and verification to implement the Ko rule.

1.6 Save and Load Functionality

1.6.1 Save

There is a separate menu dedicated for saving. In that menu, available save files are displayed in cards on the left side. Upon clicking, the card being clicked will be highlighted, and the board state stored within the file corresponding to the card is displayed to the right of the game window. A confirmation button also appears under the board displayment screen. If the user clicks the button, the game will proceed to overwrite that save file.

1.6.2 Load

There is a separate menu dedicated for loading. In that menu, available saved files are displayed in cards on the left side. Upon clicking, the card being clicked will be highlighted, and the board state stored within the file corresponding to the card is displayed to the right of the game window as a preview. A confirmation button also appears under the board displayment screen. When users click on the button, the board immediately load the respective save file.

1.7 Technology stack

The game's logic, game state management and GUI(graphic user interface) are written in C and C++, using the SFML library for graphics. The current board state is written to a plain text file in a custom notation, including the board's size, komi, and the list of moves in order.

Chapter 2

Design document

2.1 System Requirements

This application has been developed and tested primarily on the **Windows** operating system. The application requires specific Dynamic Link Libraries (DLLs) from the MinGW compiler and SFML library. These are included in the distribution folder to ensure the application runs without requiring the user to install development tools globally.

2.2 Build and Execution Instructions

The project is built using **CMake** and the **SFML 2.6.1** multimedia library. The distribution package includes both the full source code and a pre-compiled executable for immediate testing.

2.2.1 Method 1: Building from Source

If you wish to compile the project from scratch, your system must have a C++ compiler, CMake, and SFML 2.6.1 installed.

Prerequisites

- **C++ Compiler:** MinGW-w64 or Microsoft Visual C++.
- **Build Tool:** CMake (Version 3.20 or newer).
- **Library:** SFML 2.6.1 (Must match your compiler version).

Step-by-Step Build Instructions

1. **Install SFML:**
   - Download SFML 2.6.1 GCC/MinGW 64-bit from sfml-dev.org.
   - Extract it to a known location (e.g., `C:/SFML`).
2. **Configure CMake:** Open the `CMakeLists.txt` file in the root directory. Ensure the `find_package` or `SFML_DIR` path points to your installation of SFML.
3. **Build using Terminal/PowerShell:** Open PowerShell inside the root `GoGame_CS160` folder and run the following commands:

```
\\ Create a build directory
mkdir build
cd build

\\Configure the project (Point CMAKE_PREFIX_PATH to your SFML folder)
cmake -G "MinGW␣Makefiles" -DCMAKE_PREFIX_PATH="C:/SFML" ..

\\Compile the code
cmake --build .
```

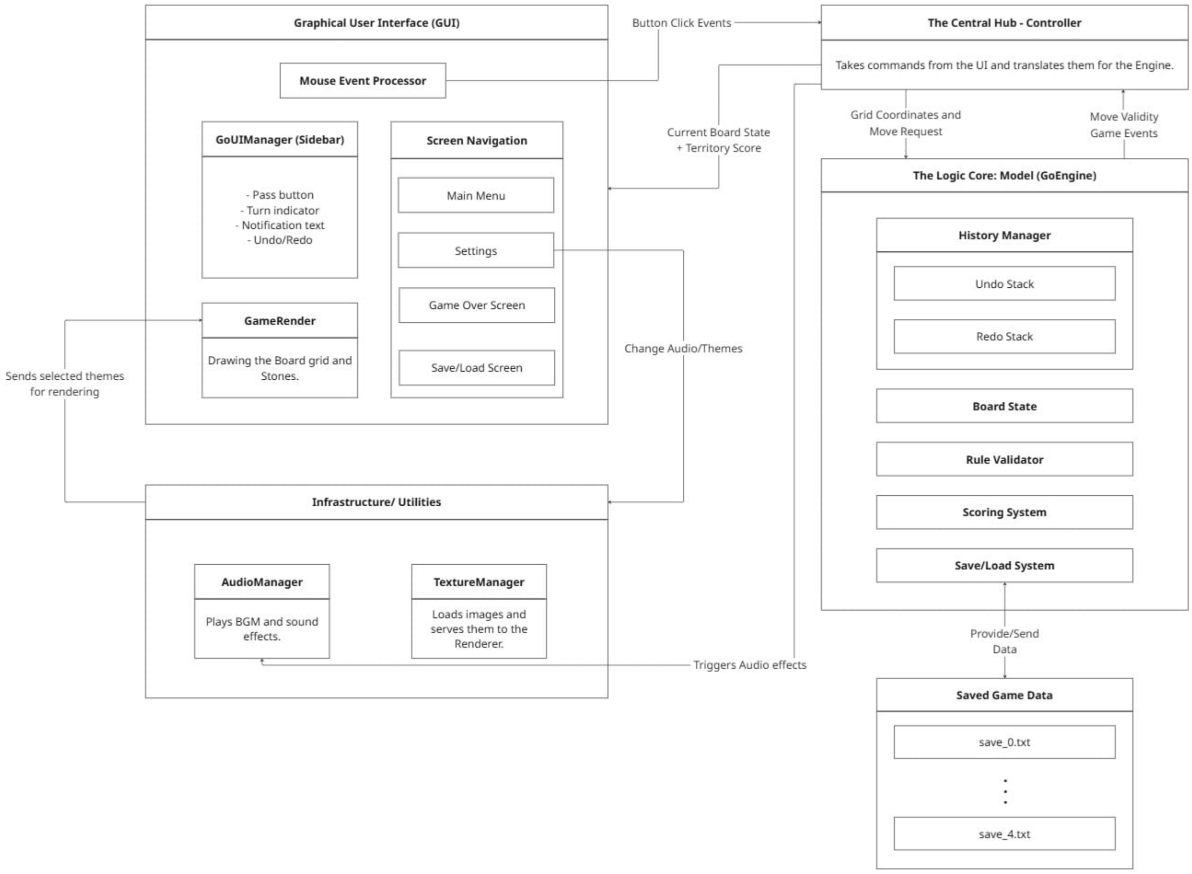
4. **Setup Assets and DLLs:** Before running the newly built executable (which will appear in the `build` folder), you must:
   - Copy the `assets` folder from the project root into your `build` folder.
   - Copy the SFML `.dll` files (from `C:/SFML/bin`) into your `build` folder.
5. **Run:** Type `./GoGame_CS160.exe` in the terminal to start the game.

2.2.2 Method 2: Building using CLion (Recommended IDE)

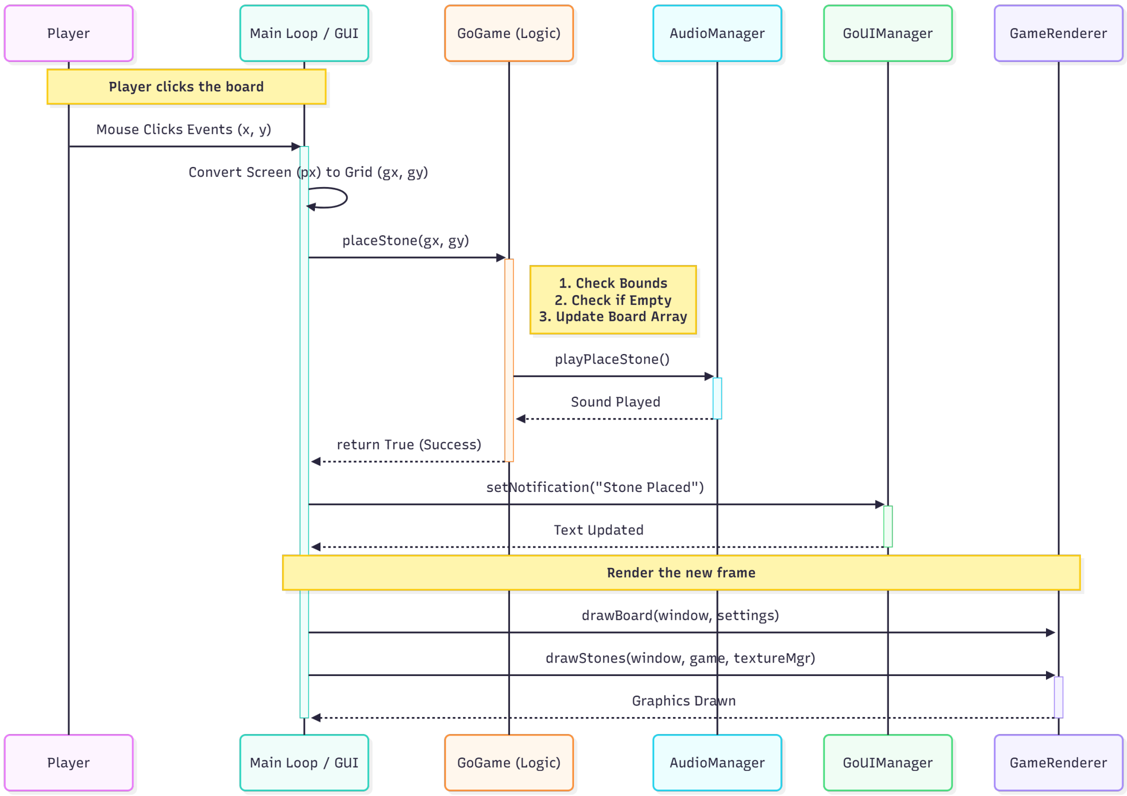
If you have the CLion IDE installed:

1. Open the `GoGame_CS160` folder as a project in CLion.
2. Go to **File** → **Settings** → **Build, Execution, Deployment** → **CMake**.
3. In the "CMake Options" field, add: `-DCMAKE_PREFIX_PATH=C:/path/to/your/SFML`.
4. Click the **Build** (Hammer icon) or **Run** (Green Play icon) button. CLion will automatically handle the compilation and asset management.

2.3 Overview


Graphical User Interface (GUI)
Button Click Events
The Central Hub - Controller
Mouse Event Processor
Takes commands from the UI and translates them for the Engine.
GoUIManager (Sidebar)
Screen Navigation
Current Board State
+ Territory Score
Grid Coordinates and
Move Request
Move Validity
Game Events
- Pass button
- Turn indicator
- Notification text
- Undo/Redo
Main Menu
The Logic Core: Model (GoEngine)
Settings
History Manager
Undo Stack
Game Over Screen
Redo Stack
GameRender
Sends selected themes
for rendering
Drawing the Board grid and
Stones.
Change Audio/Themes
Save/Load Screen
Board State
Rule Validator
Infrastructure/ Utilities
Scoring System
Save/Load System
AudioManager
TextureManager
Plays BGM and sound
effects.
Loads images and
serves them to the
Renderer.
Provide/Send
Data
Triggers Audio effects
Saved Game Data
save_0.txt
save_4.txt

2.4 Player's Move Logic



At the beginning of a player's turn, the current board configuration is serialized and pushed into the state manager (Undo Stack), as demonstrated in the initial actions of the diagram. The system then awaits user input via the mouse.

Unlike moving pieces in Chess, Go involves placing new stones. The logic proceeds as follows:

1. The player hovers the mouse over the board; the system calculates the nearest grid intersection.
2. A "ghost stone" is rendered to preview the move.
3. Upon clicking an intersection, the system validates the move (checking if the spot is empty and ensuring no rules like *Suicide* or *Ko* are violated).
4. If valid, the stone is placed, the board state is updated, and the turn passes to the opponent.

2.5 User Interface

2.5.1 Consistent Theme

The game adopts a classic aesthetic by default, utilizing a wood-textured board with high-contrast black and white stones to simulate a traditional Go experience.

To accommodate player preferences and lighting conditions, the application also provides alternative themes (e.g., a Ocean or Galaxy Mode), which can be toggled in the Settings menu.

2.5.2 Clarity and Simplicity

Priority is placed on the visibility of the board state. The design employs a distinct color palette to differentiate between black stones, white stones, and the board grid. Visual distractions are minimized by avoiding excessive animations or decorative elements that do not directly serve gameplay utility.

2.5.3 Minimalism in Navigation

The application implements an intuitive navigation menu containing only essential options (e.g., Play, Load, Settings). Clutter is reduced by grouping advanced or less frequently used features under expandable sub-menus.

Critical action buttons, such as “Pass,” “Undo,” and “Redo,” are clearly labeled and positioned adjacent to the board for easy access without overwhelming the screen space.

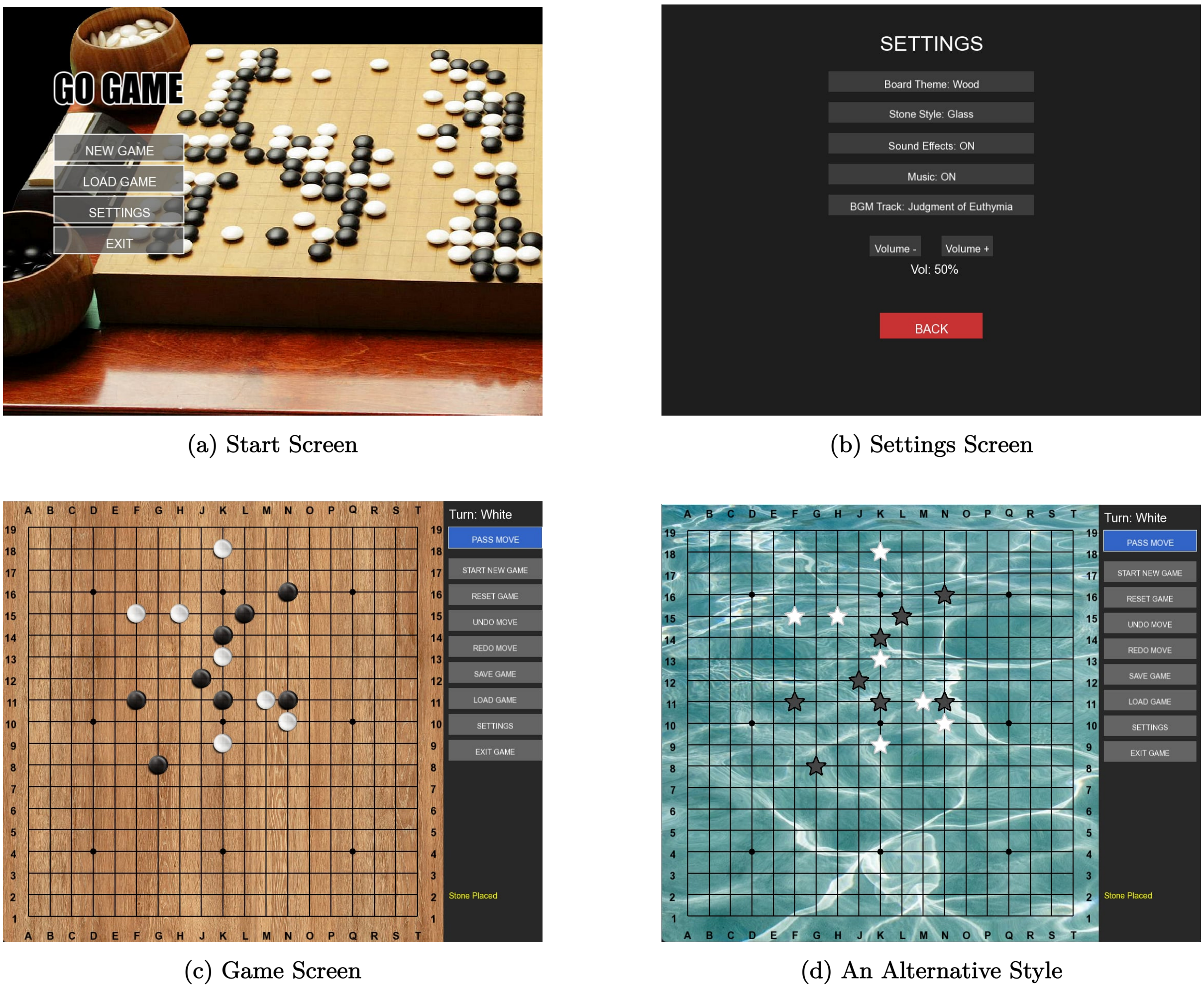
2.5.4 Feedback and Interactivity

The game provides immediate visual feedback to enhance user interaction:

- **Ghost Stones:** semi-transparent stones are rendered at intersections where the move is pre-validated as legal. This suppresses the ghost stone on illegal intersections, effectively serving as a direct visual guide for legal stone placement.
- **Turn Indicators:** The UI clearly highlights whose turn it is.
- **Game State Notifications:** Critical game events, such as the number of consecutive passes or the capture of stones, are displayed via non-intrusive notifications.

Transitions between different game states (e.g., loading a save file or performing an Undo operation) are designed to be smooth and instantaneous.

Ensure smooth and unobtrusive transitions between different game states (e.g., Undo, Redo, Load).



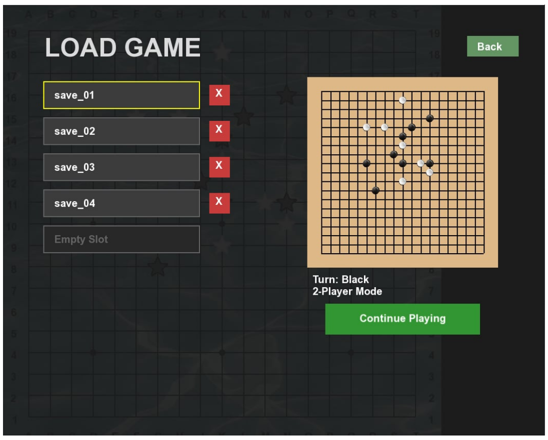
(a) Start Screen

(b) Settings Screen

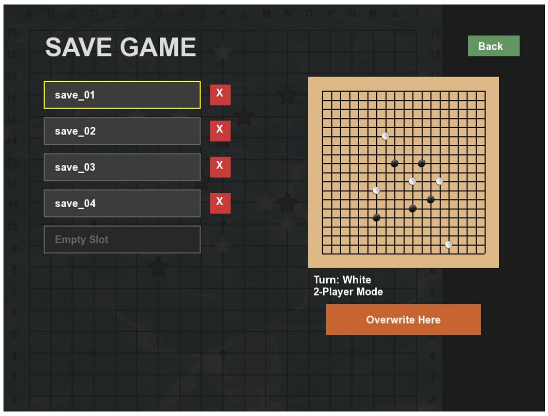
(c) Game Screen

(d) An Alternative Style

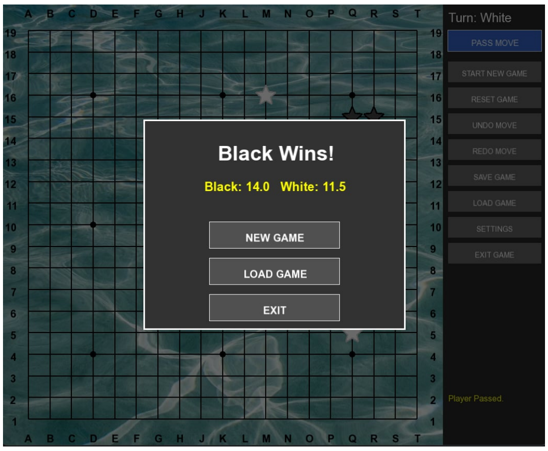
Figure 2.1: Application User Interface Screens



(e) Load Screen



(f) Save Screen



(g) Game Over Screen

Figure 2.1: Application User Interface Screens (Continued)

Chapter 3

Game Development Tutorial

3.1 Introduction

This document serves as a comprehensive guide for setting up a C++ game development environment. It details the necessary tools, libraries, and architectural patterns required to build a robust Graphical User Interface (GUI) application. Adhering to modern C++ standards, this tutorial utilizes a strict separation of concerns, ensuring that the high-level application flow is distinct from low-level implementation details.

3.2 System Requirements and Environment

The development environment for this project is designed to be cross-platform, though it has been primarily tested on **Windows 10/11**. To replicate the development setup, the following components are required.

3.2.1 Compilers and Build Tools

A C++17 compliant compiler is necessary to utilize modern language features such as ‘std::optional‘, structured bindings, and improved memory management smart pointers.

- **Windows:** MinGW-w64 (GCC 7.3+) or MSVC (Visual Studio 2019+).
- **Linux:** GCC or Clang installed via package manager.
- **macOS:** Clang (via Xcode Command Line Tools).
- **CMake:** Version 3.10 or higher is required for build automation. CMake allows us to generate makefiles or project files (like ‘.sln‘) independent of the specific IDE being used.

3.2.2 Integrated Development Environments (IDEs)

While any text editor can be used, the following IDEs provide excellent integration with CMake and C++:

- **Visual Studio Code:** Lightweight, with the C++ and CMake Tools extensions.
- **CLion:** A powerful JetBrains IDE with native CMake support.
- **Visual Studio Community:** The industry standard for Windows C++ development.

3.3 Useful Libraries for C++ Game Development

Selecting the right libraries is critical for efficiency. Instead of reinventing the wheel (e.g., writing raw OpenGL code or window handling logic), we leverage established open-source libraries.

3.3.1 Primary Library: SFML (Simple and Fast Multimedia Library)

For this tutorial, we utilize **SFML 2.6.1**. SFML provides a simple interface to the various components of your PC, offering modules for:

- **System:** Time management, threads, and data streams.
- **Window:** Managing application windows and OpenGL contexts.
- **Graphics:** Hardware-accelerated 2D graphics.
- **Audio:** Sound effects and music playback.
- **Network:** Socket-based communication.

3.3.2 Other Recommended Libraries

In a production-grade C++ game engine, you might also consider integrating:

- **Dear ImGui:** An immediate-mode GUI library, perfect for building debug tools and level editors overlaying your game.
- **Lua:** A lightweight scripting language often embedded in C++ engines to allow designers to write game logic without recompiling the core engine.
- **Boost:** A massive collection of portable C++ source libraries that extend the functionality of the standard library.
- **Box2D:** A 2D physics engine for simulating rigid bodies.

3.4 Architectural Design

To ensure maintainability and scalability, this project adheres to a strict **Header-Implementation** separation. The codebase is organized to separate the high-level application flow from the low-level implementation details.

3.4.1 The Game Loop Pattern

The core of any game is the "Game Loop." Unlike standard software that reacts only to input, a game must run continuously.

1. **Process Input:** Handle user keystrokes and mouse movements.
2. **Update:** Calculate physics, movement, and game logic.
3. **Render:** Draw the current state of the world to the screen.

3.5 Implementation Details

This section provides a detailed walkthrough of the starter code.

3.5.1 Step 1: Build Configuration (CMakeLists.txt)

The ‘CMakeLists.txt‘ file defines how our code is compiled. It instructs CMake to find the SFML package and link its libraries to our executable.

Listing 3.1: CMakeLists.txt Configuration

```
cmake_minimum_required(VERSION 3.10)
project(SFMLGameTutorial VERSION 1.0)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED True)

# Locate SFML Package
# Ensure SFML_DIR is in your environment variables if not found automatically
find_package(SFML 2.6 COMPONENTS graphics window system REQUIRED)

set(SOURCES main.cpp Game.cpp Game.h)

add_executable(GameApp ${SOURCES})

# Link SFML Libraries to the executable
target_link_libraries(GameApp sfml-graphics sfml-window sfml-system)

# Copy assets to build directory
file(COPY ${CMAKE_SOURCE_DIR}/assets DESTINATION ${CMAKE_BINARY_DIR})
```

3.5.2 Step 2: The Engine Interface (Game.h)

The header file acts as the contract for our class. We define the ‘Game‘ class which holds the ‘sf::RenderWindow‘. We use a pointer for the window to allow for dynamic memory allocation and lifetime control.

Notice the separation of private functions (‘initVariables‘, ‘initWindow‘) from public functions. This encapsulates the initialization complexity, keeping the public interface clean.

Listing 3.2: Game.h Interface

```
#pragma once

#include <SFML/Graphics.hpp>
#include <SFML/Window.hpp>
#include <SFML/System.hpp>

class Game {
private:
    // --- Core Variables ---
    sf::RenderWindow* window;
    sf::VideoMode videoMode;
    sf::Event ev;

    // --- Game Objects ---
    sf::CircleShape player;

    // --- Private Initialization ---
```

```
    void initVariables();
    void initWindow();
    void initPlayer();

public:
    // --- Constructors / Destructors ---
    Game();
    virtual ~Game();

    // --- Accessors ---
    const bool running() const;

    // --- Core Functions ---
    void pollEvents();
    void update();
    void render();
};
```

3.5.3 Step 3: Logic Implementation (Game.cpp)

The '.cpp' file contains the actual logic.

Double Buffering: In the 'render()' function, we see the 'clear()', 'draw()', 'display()' pattern. This is known as double buffering. The computer draws the frame on a hidden "back buffer" while the user sees the "front buffer." Only when drawing is complete do we swap them ('display()'). This prevents screen tearing and flickering.

Listing 3.3: Game.cpp Implementation

```
#include "Game.h"

// --- Private Functions ---

void Game::initVariables() {
    this->window = nullptr;
}

void Game::initWindow() {
    this->videoMode.height = 600;
    this->videoMode.width = 800;

    // Create the window with title and default style
    this->window = new sf::RenderWindow(
        this->videoMode,
        "C++␣Game␣Architecture␣Tutorial",
        sf::Style::Titlebar | sf::Style::Close
    );

    this->window->setFramerateLimit(60);
}

void Game::initPlayer() {
```

```cpp
    this->player.setRadius(50.f);
    this->player.setPosition(sf::Vector2f(350.f, 250.f));
    this->player.setFillColor(sf::Color::Cyan);
    this->player.setOutlineColor(sf::Color::Blue);
    this->player.setOutlineThickness(2.f);
}

// --- Constructor / Destructor ---

Game::Game() {
    this->initVariables();
    this->initWindow();
    this->initPlayer();
}

Game::~Game() {
    delete this->window;
}

// --- Accessors ---

const bool Game::running() const {
    return this->window->isOpen();
}

// --- Functions ---

void Game::pollEvents() {
    // PollEvent pops the event from the top of the queue
    while (this->window->pollEvent(this->ev)) {
        switch (this->ev.type) {
            case sf::Event::Closed:
                this->window->close();
                break;
            case sf::Event::KeyPressed:
                if (this->ev.key.code == sf::Keyboard::Escape)
                    this->window->close();
                break;
            default:
                break;
        }
    }
}

void Game::update() {
    this->pollEvents();

    // Simple Movement Logic
    if (sf::Keyboard::isKeyPressed(sf::Keyboard::A))
        this->player.move(-5.f, 0.f);
    if (sf::Keyboard::isKeyPressed(sf::Keyboard::D))
```

```
        this->player.move(5.f, 0.f);
    if (sf::Keyboard::isKeyPressed(sf::Keyboard::W))
        this->player.move(0.f, -5.f);
    if (sf::Keyboard::isKeyPressed(sf::Keyboard::S))
        this->player.move(0.f, 5.f);
}

void Game::render() {
    // 1. Clear old frame
    this->window->clear(sf::Color(20, 20, 20));

    // 2. Draw objects
    this->window->draw(this->player);

    // 3. Display new frame
    this->window->display();
}
```

3.5.4 Step 4: The Entry Point (main.cpp)

Finally, the ‘main.cpp‘ is kept extremely minimal. It merely instantiates the ‘Game‘ engine and runs the main loop. This abstraction allows us to expand the game indefinitely without cluttering the main entry point.

Listing 3.4: main.cpp Entry Point

```
#include "Game.h"

int main() {
    // Initialize the Game Engine
    Game game;

    // Game Loop
    while (game.running()) {
        // Update state
        game.update();

        // Render application
        game.render();
    }

    return 0;
}
```

3.6 Build and Execution Instructions

With the code in place, compilation is handled via CMake.

3.6.1 Compiling the Source

1. Open a terminal in the project root.

2. Create a build directory: `mkdir build && cd build`
3. Configure the project: `cmake ..`
4. Compile: `cmake --build .`

Important Note: Do **not** move the generated '.exe' file out of the 'bin' or 'Debug' directory. The executable depends on local assets and DLLs. Moving it will result in a crash upon launch.

Chapter 4

Implementation Details

4.1 Code Organization and Structure

The codebase adheres to a modular, object-oriented structure, separating concerns across distinct classes and file pairs. This approach maximizes maintainability, reduces compilation times, and ensures logical separation between the core game logic and the graphical interface.

The project utilizes a standard CMake hierarchy, organized into the following key directories:

- **`include/`**: Stores all header files (`.h`) containing class definitions and forward declarations.
- **`src/`**: Stores all implementation files (`.cpp`) where the functionality of the classes is defined.
- **`assets/`**: Contains all external resources, including stone textures, fonts, themes, and audio files.

4.1.1 Structured Folder Hierarchy

```
GoGame_CS160/
├── CMakeLists.txt
├── assets/
│   ├── fonts/
│   ├── img/
│   └── audio/
├── include/
│   ├── Definitions.h
│   ├── GoGame.h
│   ├── AudioManager.h
│   └── ..
└── src/
    ├── main.cpp
    ├── GoGame.cpp
    ├── AudioManager.cpp
    └── ..
```

4.2 Libraries and Technology Stack

The game's logic, game state management, and Graphical User Interface (GUI) are written in **C++**. The program utilizes the **SFML** library to handle the graphic interface.

- **Primary Library: SFML 2.6.1**. Used for all graphical rendering (board, stones, UI elements), window management, event handling, and audio processing.

- **Build System: CMake**. Used to manage the cross-platform compilation process, link external libraries, and automatically handle asset copying for runtime execution.
- **Serialization:** The current board state is written to a plain text file in a custom notation, which includes the board's size, komi, and the sequential list of moves for saving and loading functionality.

4.3 Major Component Breakdown

The codebase is divided into two major functional components: **General Logic** and the **Board/Communicator Interface**.

- **General Logic (Controllers):** This component is responsible for handling all interactions between the underlying system and the game window. It enforces game rules, manages game state (save/load, point calculation), and handles interactions with the core engine. Key classes include `Board`, `GoGame`, `AudioManager`, and `SettingsScreen`.
- **Board Component (UI/View):** This component manages interactions between the user and the graphical interface. It processes mouse input, renders the board and stones, and sends necessary information (e.g., coordinates of a click) back to the General Logic component for validation and execution. Key classes include `GoUIManager`, `MainMenuScreen`, and `GameRenderer`.

Chapter 5

Testing

5.1 Testing Strategies

To ensure the stability and reliability of the Go Game application, a multi-layered testing approach was adopted, focusing on verifying individual components as well as the complete system workflow.

5.1.1 Functional Testing

The core mechanics were tested rigorously to ensure strict adherence to the rules of Go.

- **Move Validation:** Verification was performed to ensure stones cannot be placed on occupied intersections or out-of-bounds coordinates.
- **Capture Logic:** Scenarios were executed to confirm that surrounding an opponent's stone (or group of stones) correctly removes them from the board.
- **Rule Enforcement:** Specific edge cases, such as the *Ko Rule* (preventing infinite loop repetition) and *Suicide Rule*, were manually triggered to ensure the engine rejects these moves.

5.1.2 Interface and Integration Testing

This phase focused on the interaction between the user interface (GUI) and the backend logic.

- **Button Responsiveness:** All interface elements (Undo, Redo, Save, Load) were tested to ensure correct state changes occur without delay.
- **State Persistence:** The Save/Load system was tested by saving games in complex states, terminating the application, and reloading to verify that the board, turn order, and history stacks were restored accurately.
- **Visual Feedback:** The rendering of "ghost stones" was verified to ensure they appear on valid moves and are suppressed on illegal ones.

5.2 Test Results

The application demonstrated stability across the following metrics:

- **Stability:** The application operates on Windows 10 and 11 without memory leaks or crashes during extended sessions.

- **Performance:** A steady 60 FPS is maintained, with instant response times for move placement and logic calculations.
- **Data Integrity:** Save files are generated correctly in the expected text format and can be loaded reliably.

5.3 Known Issues and Limitations

Despite the successful implementation of core features, the current version of the project possesses the following limitations:

5.3.1 Lack of Artificial Intelligence (AI)

The game supports only **Local 2-Player** mode. No computer opponent (AI) is currently implemented.

5.3.2 Fixed Window Resolution

The application window is fixed at a specific resolution (1000×850 pixels). Dynamic resizing and full-screen modes are not supported. UI scaling may not be optimal on monitors with extremely high (4K) or low resolutions.

5.3.3 Platform Dependency

The build system and executables are optimized for the **Windows** environment (MinGW/MSVC). Cross-platform compilation for Linux or macOS has not been configured in the current CMake setup.

Chapter 6

Future Improvements

1. Implementation of board rotation for better user experience.
2. Implement a Player vs AI mode with 3 different levels from easy to hard.
3. Implement scalable windows that is adaptive to different screen sizes and scale levels.
4. Implement a function that make suggested dead stones glow red to help with ease of use in the counting phase.